# When RAN Intelligent Controller in O-RAN Meets Multi-UAV Enable Wireless Network

Chuan Pham, Foroutan Fami, Kim Khoa Nguyen, and Mohamed Cheriet

**Abstract**—Unmanned aerial vehicles (UAVs) are projected to be utilized in a variety of unexpected applications, including agriculture, firefighting, emergency response, intelligent transportation, and so on. Wireless communication is one of the primary facilitators in bringing UAVs into a new phase in such applications. To realize the vision in fifth-generation (5G) networks, we propose a 5G-integration of the flexible multi-UAV system and the Open Radio Access Network (O-RAN) architecture, named U-ORAN. Although different studies have been proposed to optimize the UAV trajectory and resource allocation in the radio access network (RAN), our work is the first study to investigate the benefits of adopting UAVs in the O-RAN architecture. In U-ORAN, we consider a flying base station system and propose a joint optimization problem of multi-UAV trajectory and offloading tasks (UTOT) in which UTOT can optimize the routes from users to the core network as well as resource allocation to process offloading tasks. We decompose UTOT into two sub-problems and provide learning solutions based on the multi-agent reinforcement learning and online learning methodologies, both of which are well supported by the O-RAN architecture. Our intensive numerical simulations show that the proposed approaches outperform in a variety of settings and validation scenarios.

**Index Terms**—Unmanned aerial vehicles (UAVs), open radio access network (O-RAN), resource allocation, reinforcement learning, online learning, task offloading

## 1 Introduction

Unmanned aerial vehicles (UAVs) have become a promising and prevalent solution to deal with a large number of connections in the Internet of Things (IoT) networks, such as monitoring devices, smart traffic sensors, smart farming sensors, and so on, as their use in various applications has grown rapidly. Moreover, 5G and future mobile services impose stringent requirements, such as millisecond latency or up to 1000 × bandwidth per unit area which requires BSs more resources and higher availability. Among several solutions in the up-to-the-minute works, the use of the unmanned aerial vehicle base stations (UAV-BSs) has been much attractive in research and industry in order to satisfy the never-ending thirst of heterogeneous demand in several unplanned and temporary situations which cannot be coped with the existing architecture [1]. This is mainly because of the following reasons. First, the fast and potential mobility of UAVs can facilitate a rapid and flexible infrastructure to assist in special cases, such as temporary events or disasters in which the current infrastructure is unable to meet user demand due to common issues, e.g., power outages or high user demand [2]. Second, the extra benefit of line-of-sight (LOS) communication makes UAV systems more capable of ensuring stable connections to ground users than traditional cell tower-based methods, according to [3].

Last but not least, according to [4], [5], recent UAV models can carry a large payload, for example, Hercules 20 is able to carry up to 15 kg (33 lbs) payload flying for 15 minutes with high speed (25 m/s). Consequently, high-performance onboard computing resources for UAVs can be enabled to service ground users [6], [7], [8]. These qualities distinguish UAVs from typical base stations, making them really feasible for low-cost deployment in unexpected operation circumstances [1], [3].

To facilitate the terrestrial network, we rely on the recently introduced architecture of Open Radio Access Network (O-RAN) [9] to deploy a flying 5G O-RAN architecture as shown in Fig. 1. Such a system can provide 5G services, enhance the quality of services (QoS) and assist the traditional network system for a temporary period. O-RAN opens many new opportunities but also faces new challenges regarding network design, deployment, configuration, optimization, automation, and operation. Unlike traditional RAN systems, O-RAN components such as the Open Radio Unit (O-RU), Open Distributed Unit (O-DU), and Open Centralized Unit (O-CU) are allowed to be distributively and flexibly deployed. Open interfaces are defined to connect these components using different strategies and with respect to specific objectives (e.g., QoS, security). Consequently, U-ORAN requires optimized routing mechanisms to operate the system. In addition, using UAVs as O-RUs allows to quickly deploy a 5G network to assist the terrestrial network in a temporary period in which computing resources can fly closer to users to meet 5G stringent constraints. Last but not least, to efficiently control UAV-BSs, intelligent mechanisms can have strong support in

• *The authors are with the Synchromedia - École de Technologie Supérieure, Université du Québec, Québec H3C1K3, Canada. E-mail: pchuanvn@gmail.com, Foroutan.Fami.1@ens.etsmtl.ca, {Kim-Khoa.Nguyen, mohamed.cheriet}@etsmtl.ca.*

*Manuscript received 19 October 2021; revised 21 March 2022; accepted 2 July 2022. Date of publication 29 July 2022; date of current version 6 September 2023.*
*This work was supported in part by Mitacs, in part by Ciena, and in part by ENCQOR under Grant IT13947.*
*(Corresponding author: Chuan Pham.)*
*Recommended for acceptance by Y. Yang.*
*Digital Object Identifier no. 10.1109/TCC.2022.3193939*

2168-7161 © 2022 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.
Authorized licensed use limited to: National Yang Ming Chiao Tung University. Downloaded on October 19,2025 at 06:42:59 UTC from IEEE Xplore. Restrictions apply.

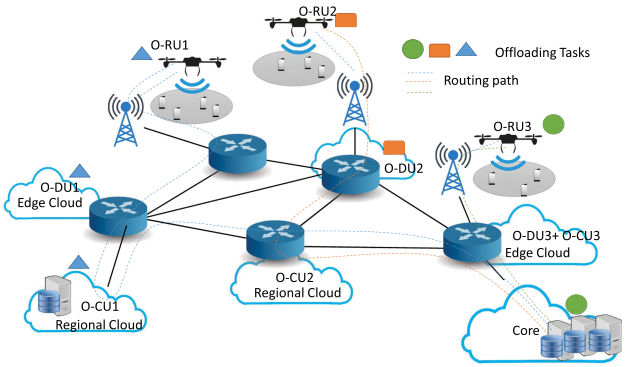



Fig. 1. Overview architecture of U-ORAN.

implementation and deployment by using the intelligent O-RAN components including non-real-time and near-real-time RAN Intelligent Controller (RIC).

As presented in [10], [11], [12], [13], [14], the multi-objective joint optimization of trajectory and resource allocation in the UAV communication system has been attractive in both research and practice. By joint optimizing the controlling UAVs and resource allocation decisions (e.g., user associations, level of power transmission, the amount of CPU to server offloading tasks) [13], UAV-BSs become a potential solution to process offloading tasks [14], [15], [16], [17]. Thanks to their flexibility with heavy payload capacity, UAV-BSs can be more proximate to users than the conventional tower base stations to achieve a higher communication quality.

In addition, O-RAN supports enable the implementation of UAV-BSs [9]. RIC in O-RAN can enable optimization and learning solutions to employ and control UAV-BSs in order to extend not only the coverage area but also the computing resource availability. Offloading tasks have more opportunities to be served with higher quality rather than relying just on edge/cloud servers. However, deploying UAVs in the O-RAN architecture also raises many challenges because of flexible and distributed computing resource units and the movement of UAVs. Such issues that cannot be solved by using traditional methods require a new model and solution to cope with.

Different from prior works [15], [16], [17], our study is *the first contribution introducing learning solutions for the flying O-RAN architecture to optimize not only the UAV-BS trajectory but also distributed resource units to serve offloading tasks.* In fact, these two issues are highly interdependent. An optimized UAV-BS trajectory can improve the system's coverage capacity but also enhance the availability of computing resources in networks. Making an appropriate resource allocation decision can optimize resource allocation, help control UAVs in terms of user association decisions and energy management consumed by flying and processing offloading tasks. Furthermore, our model takes into account the combination of UAV-BSs and the O-RAN architectures allowing for varied routing from end-users to compute units to perform offloading activities though it adds more complexity to the joint optimization problem.

Summarily, the contributions of our work are listed as follows:

- We study *a system model of U-ORAN* and formulate *a joint UAV trajectory and offloading-task allocation optimization problem*. Based on the distributed architecture of O-RAN, we aim to jointly optimize the movement of UAV-BSs and resource allocations in order to meet offloading constraints. Specifically, we make *association decisions* between UAV-BSs and users to optimize *UAV-BS's trajectories* and the *offloading decisions* in which offloading tasks will be efficiently served by appropriate O-RAN elements (O-RU, O-DU, and O-CU). To the best of our knowledge, the *proposed formulation of this joint problem in U-ORAN* is not available in the state-of-the-art works.
- We design a *control loop* that defines flows to operate the UAV-BSs and manages the distributed elements of O-RAN for serving offloading tasks.
- To solve the proposed optimization problem, UTOT, we decompose it into sub-problems and then solve them by *the multi-agent reinforcement learning framework [18] and the online learning and matching method [19]*. Our proposed methods allow optimizing the system within a time horizon.

The rest of this paper is organized as follows. In Section 2, we discuss the state of the art. Section 3 introduces the system model and the problem formulation. Section 4 presents the designed algorithms in centralized and distributed manners. We present the simulation to evaluate the proposed methods in Section 5. Finally, we conclude the paper in Section 6.

## 2 Related Work

In this section, we briefly present selected prior works related to task offloading and resource allocation with the UAV-assisted system.

### 2.1 UAV-Assisted Systems for Offloading Tasks

There are plenty of works focusing on the optimized resource allocation for offloading tasks with one or many UAV-BSs. As shown in [20], the authors proposed a game theory-based approach to optimize the multi-objective including energy, latency, and computing cost. However, these authors did not consider a flexible sharing method to perform tasks among UAVs and edge/cloud servers. They also conduct a simple system with only the battery constraint meanwhile in practice, offloading tasks need more considered aspects, such as latency of processing and propagation, resource constraints, and the routing mechanism as well. Reinforcement learning has been regarded as a promising method to deal with resource allocation and offloading tasks in the UAV-assisted system. In another work, [21] studied artificial intelligence (AI) in the UAV-data offloading process under a multi-server Mobile Edge Computing (MEC) system by using game theory and reinforcement learning approaches. The authors proposed a non-cooperative game to determine partial data offloaded MEC server while the remaining is performed by UAVs. They simplified the model by ignoring the routing issues to forward the offloading tasks to MEC servers. However, it is one of the main reasons degrading the performance of services since a static routing mechanism might not satisfy all the time, especially in the flexible environment of UAVs and the distributed network of O-RAN. Considering a different aspect of offloading tasks, the authors in [14] presented a method

Authorized licensed use limited to: National Yang Ming Chiao Tung University. Downloaded on October 19,2025 at 06:42:59 UTC from IEEE Xplore. Restrictions apply.

to migrate tasks from UAVs to edge or cloud data centers. But they ignored the energy issue in their method, such an important aspect of the UAV-BS system.

### 2.2 Task Offloading in RAN

Computation offloading in the radio access network (RAN) can release the tension between computation-intensive tasks and computation-limited mobile devices to enrich the quality of services. Another reason that makes new architectures of RAN such as D-RAN, C-RAN, and V-RAN become potential to process offloading tasks is the distributed computing resources tiered with access points, base stations, edge, regional and core cloud centers to assist mobile devices in processing their tasks in parallel [22], [23], [24]. The authors in [22] illustrated an efficient mechanism to manage, schedule, and process offloading tasks based on the distributed architecture of Fog RAN. [25] considered a granularity level of offloading tasks to propose an offloading mechanism based on C-RAN. However, to reap the benefits of the new evolution of the distributed RAN architecture, the routing and allocating computing resource issues should be judiciously considered since they have a significant impact on the latency and task execution delay. These problems become more difficult in the disaggregated architecture of O-RAN in which resource units, such as O-CU, O-DU, and O-RU, are more flexible to be deployed and scaled in the system.

The major contribution of our work is to develop a joint optimization of multi-UAV trajectory and offloading task allocation underlying the O-RAN architecture. To the best of our knowledge, our work is one of the first works studying the flying base stations to serve offloading tasks in O-RAN where all the distributed elements of O-RAN are considered. Despite the fact that numerous research has been conducted on optimizing UAV trajectory using the reinforcement learning approach, none of the current work has investigated the coupling of the UAV system and the disaggregated architecture of O-RAN.

## 3 System Model and Problem Statement

### 3.1 System Model

We consider a scenario in which flying base stations are deployed based on O-RAN architecture to serve terrestrial users in a specific area during a temporary period $T$ in order to meet diverse and stringent 5G requirements. Each UAV-BS is equipped with an O-RU to serve mobile users. The system model of U-ORAN is depicted in Fig. 1. Thanks to the high capacity of recent UAVs, we assume they can carry a container-based network function [8]. However, the on-boarding computation resources of a UAV are constrained by its limited battery capacity. Therefore, UAV-BSs can play a role as O-RUs and process simple tasks described in O-RAN specifications [9]. Connecting to O-RUs, an O-RAN distributed unit (O-DU) is a logical node that hosts RLC/MAC/High-PHY layers based on a lower layer functional split in O-RAN. The O-DU handles a set of eNB/gNB functions controlled by the O-RAN Central Unit (O-CU) that is used to run the radio resource control (RRC) and Packet Data Convergence Protocol (PDCP) layers. According to the O-RAN specifications, an O-CU can connect to multiple O-DUs in order to support multiple gNBs.

TABLE 1
Notations

| | |
|---|---|
| | Parameters of Network Topology |
| $\mathcal{V}$ | The set of UAV-BSs. |
| $\mathcal{D}$ | The set of O-DUs. |
| $\mathcal{C}$ | The set of O-CUs. |
| $\mathcal{R}$ | The set of O-CUs. |
| $\mathcal{N}$ | The set of computing resource nodes. |
| $\mathcal{I} = \{i\}$ | The set of users indexed by $i$. |
| $\mathcal{I}_v$ | The set of users associated with UAV $v$. |
| $\mathcal{E}$ | The set of links between resource nodes. |
| $L \times L$ | The considered area. |
| $T$ | The considered period. |
| | **Parameters of the UAV model** |
| $o_v(t)$ | The 3D-coordinate of UAV $v$ at time $t$ with $(x_v(t), y_v(t), h_v(t))$. |
| $o_i(t)$ | The 3D-coordinate of user $i$ at time $t$ with $(x_i(t), y_i(t), 0)$. |
| $\kappa_v$ | The payload of UAV $v$. |
| $\Delta t$ | Duration of a time slot. |
| $E_v^{\text{fly}}$ | The flying energy consumption of UAV $v$. |
| $P_{iv}^{\text{LoS}}$ | The LOS probability of a communication between user $i$ and UAV $v$. |
| $\bar{L}_{iv}^{\text{loss}}$ | The average path loss. |
| $\eta_{\text{LoS}}, \eta_{\text{NLoS}}$ | The average additional loss. |
| $P_i$ | The transmission power of user $i$. |
| $\gamma_{iv}$ | The UAV-user SNR. |
| $\theta_v$ | The elevation angle. |
| $d_{iv}$ | The distance between UAV $v$ and user $i$. |
| $w_{iv}$ | The UAV-user data rate. |
| $B$ | The communication bandwidth. |
| $(\lambda_i, c_i, L_i)$ | Data size, computing cycles and latency threshold required for processing one bit data of task $i$. |
| $\mu_n$ | The service rate capacity of node $n$. |
| $C_{mn}$ | The bandwidth capacity of link $(mn)$. |
| $L_{mn}$ | The average latency of link $(mn)$. |
| $\alpha, \beta$ | The weighted parameters. |
| $\bar{E}_v$ | The battery capacity of UAV $v$. |
| | **Variables** |
| $x_{in}(t)$ | The probability variable of placing task $i$ on node $n$ at time $t$. |
| $w_{(mn)}^i(t)$ | The data rate routing variable of task $i$ on link $(mn)$ at time $t$. |

We consider a set of containers to virtualize O-RAN elements (e.g., O-RUs, O-DUs, and O-CUs) deployed in UAVs and distributed computing nodes of the network as shown in Fig. 1. Given the set of containers with different functions, we aim to determine the optimal UAV-user associations and task offloading scheme in order to serve ground users efficiently.

Here, we use $\mathcal{D}$, $\mathcal{C}$ and $\mathcal{R}$ to denote the sets of O-DU, O-CU, and containers on core data centers, respectively. Furthermore, we denote the set of UAV-BSs by $\mathcal{V}$. Hence, we present the set of computing resources in our O-RAN-based model by $\mathcal{N} = \mathcal{V} \cup \mathcal{D} \cup \mathcal{C} \cup \mathcal{R}$. We aim to optimize the performance of serving offloading tasks including i) *controlling UAV-BSs to guarantee the quality of communication channels between UAV-BSs and users*, and ii) *efficiently distributing offloading tasks to appropriate O-RAN elements according to the current association*. All the notations in our problem formulation are presented in Table 1

Authorized licensed use limited to: National Yang Ming Chiao Tung University. Downloaded on October 19,2025 at 06:42:59 UTC from IEEE Xplore. Restrictions apply.

## 3.2 Problem Formulation

### 3.2.1 UAV Model

We first consider deploying a set $\mathcal{V}$ of UAV-BSs in an area with $L \times L$ square during a specific period $T$ in order to serve for a set $\mathcal{I} = \{i\}$ of users on the ground. According to [26], the energy consumption of UAV-BS $v$ is calculated by

$$E_v^{\text{fly}} = \sum_{t=1}^{T-1} 0.5\kappa_v \frac{||o_v(t) - o_v(t-1)||}{\Delta t}, \tag{1}$$

where $\kappa_v$ is the payload of UAV-BS $v$ and $o_v(t) = (x_v(t), y_v(t), h_v(t)), 0 \le x_v(t), y_v(t) \le L$ is the coordinate of UAV-BS $v$ at time $t$.

With a higher elevation of UAVs compared to the traditional towers, the LOS component, and Non-LOS component [3] create a special air-to-ground channel between a UAV and a user which is different from conventional tower-based communication. According to [10], the probability of LOS of a communication channel between a UAV $v$ and a user $i$ is computed as follows: $P_{iv}^{\text{LoS}} = \frac{1}{1+\alpha\exp(-\psi_1[\theta-\psi_2])}$, where $\psi_1$ and $\psi_2$ are terrain parameters depending on the environment (e.g., urban, rural) and $\theta = \arctan(h_v/d_{iv})$ with UAV-BS-user distance $d_{iv}$. Based on $P_{iv}^{\text{LoS}}$, the average path loss from a UAV to a user can be calculated by

$$\begin{cases} \bar{L}_{iv}^{\text{loss}} = P_{iv}^{\text{LoS}}(\theta) L_{iv}^{\text{LoS}}(d_{iv}) + [1 - P_{iv}^{\text{LoS}}(\theta)] L_{iv}^{\text{NLoS}}(d_{iv}) \\ L_{iv}^{\text{LoS}} = 20\log(4\pi f d_{iv}/c) + \eta_{\text{LoS}} \\ L_{iv}^{\text{NLoS}} = 20\log(4\pi f d_{iv}/c) + \eta_{\text{NLoS}}, \end{cases}$$

where $\eta_{\text{LoS}}$ and $\eta_{\text{NLoS}}$ are the average additional loss that depends on the environment. Since we consider a specific area, these environment parameters (e.g., $\eta_{\text{LoS}}$, $\eta_{\text{NLoS}}$, $\psi_1$ and $\psi_2$) are static and can be obtained by performing some measurements [27], [28], [29] before deploying UAVs.

To measure the signal quality that each user could experience, we use $\gamma_{iv}$ to denote the signal-to-noise-ratio (SNR) level for the UAV-user. Hence, we have $\gamma_{iv} = (P_i - \bar{L}_{iv}^{\text{loss}})/N_0$, where $P_i$ is the transmission power of user $i$ and $N_0$ is the power of background noise. Based on Shannon's calculation, we can obtain the data rate by $w_{iv} = B\log_2(1 + \gamma_{iv})$, where $B$ is the communication bandwidth.

We denote a task offloaded from a user by a tuple $(\lambda_i, c_i, L_i)$ where $\lambda_i$ is the data size, $c_i$ is the amount of computing resources requested by user $i$ (i.e., the number of the required CPU cycles for executing 1-bit of input data) and $L_i$ is the tolerant latency of task $i$. Hence, if user $i$ communicates with UAV $v$ to offload its task, it requires a serving time by $\frac{\lambda_i}{w_{iv}}, \forall i \in \mathcal{I}, \forall v \in \mathcal{V}$.

### 3.2.2 O-RAN Computing Resource Model

To simplify our analysis, we consider that each user requests a single task. Hence, given the current UAV's locations at time $t$, there is a set $\mathcal{I}$ of tasks requested by users. O-RAN elements (O-RU, O-DU, O-CU, and core network containers) are connected as a network graph $(\mathcal{N}, \mathcal{E})$, where $\mathcal{E}$ is the set of links. In order to make the resource allocation decision, we define $x_{in}(t)$ as a probability variable to represent the probability of task $i \in \mathcal{I}$ that is executed at container $n \in \mathcal{N}$ and $w_{(mn)}^i(t), \forall m \ne n \in \mathcal{N}$ is the routing data rate of task $i$ on physical link $(mn)$ at time $t$.

With the aforementioned notations, we consider the following constraints and the objective function.

*Resource Capacity.* A node can only serve offloading tasks depending on its availability of computing resources. Thus, if task $i$ is executed by container $n$, the allocation has to satisfy the following resource constraint

$$\sum_{i\in\mathcal{I}} \lambda_i c_i x_{in}(t) \le \mu_n, \forall n \in \mathcal{N}, \tag{2}$$

where $\mu_n$ is the service rate of container $n$.

*Physical Link Capacity.* We consider the link capacity constraint when the offloading request is routed from device $i$ to an executed container as follows:

$$\sum_{i\in\mathcal{I}} w_{(mn)}^i(t) \le C_{mn}, \forall (mn) \in \mathcal{E}, t = 1, 2, \ldots, T. \tag{3}$$

*Latency Constraints.* Considering the processing capacity of container $n$, we formulate the processing latency based on the M/M/1 queueing model as follows:

$$L_n^{\text{proc}} = \frac{1}{\mu_n - \sum_{i\in\mathcal{I}} \lambda_i c_i x_{in}(t)}, \forall n \in \mathcal{N}. \tag{4}$$

In addition, we take into account the propagation delay when the offloading task $i$ is routed from user $i$ to the executing container. Given the average latency $L_{mn}$ of the link $(mn)$, we can obtain the cumulative propagation delay by $L_i^{\text{trans}} = \sum_{(mn)\in\mathcal{E}} \phi(w_{(mn)}^i(t)) L_{mn}$, where $\phi(.)$ is the binary function defined by $\phi(u) = \begin{cases} 1, & \text{if } u > 0, \\ 0, & \text{if } u = 0. \end{cases}$ For simplicity, we assume that the user data is routed through a single path from the source to the destination.

### 3.2.3 Objective Function and Problem Formulation

In this work, our objective is to maximize the total network utility that can be translated into resource allocations (e.g., computing resources and link data rate), subject to the aforementioned constraints. We denote $u_i(.)$ as the utility function for each offloading task $i$ depending on *the selected container* $n$ and *routing data rate* as follows

$$u_i(\boldsymbol{x_i}, \boldsymbol{w_i}) = \alpha \sum_{n\in\mathcal{N}} P_{in}\lambda_i c_i x_{in}(t) + \beta \sum_{(mn)\in\mathcal{E}} w_{(mn)}^i(t), \tag{5}$$

where $P_{in}$ is the given probability of container $n$ that can serve task $i$, $\alpha$ and $\beta$ are coefficient parameters to convert these aspects (i.e., resource utilization and routing data rate) into the same unit.

Hence, we formulate the joint multi-UAV Trajectory and Offloading Task allocation optimization (UTOT) problem for the UAV-ORAN based architecture as follows:

Authorized licensed use limited to: National Yang Ming Chiao Tung University. Downloaded on October 19,2025 at 06:42:59 UTC from IEEE Xplore. Restrictions apply.

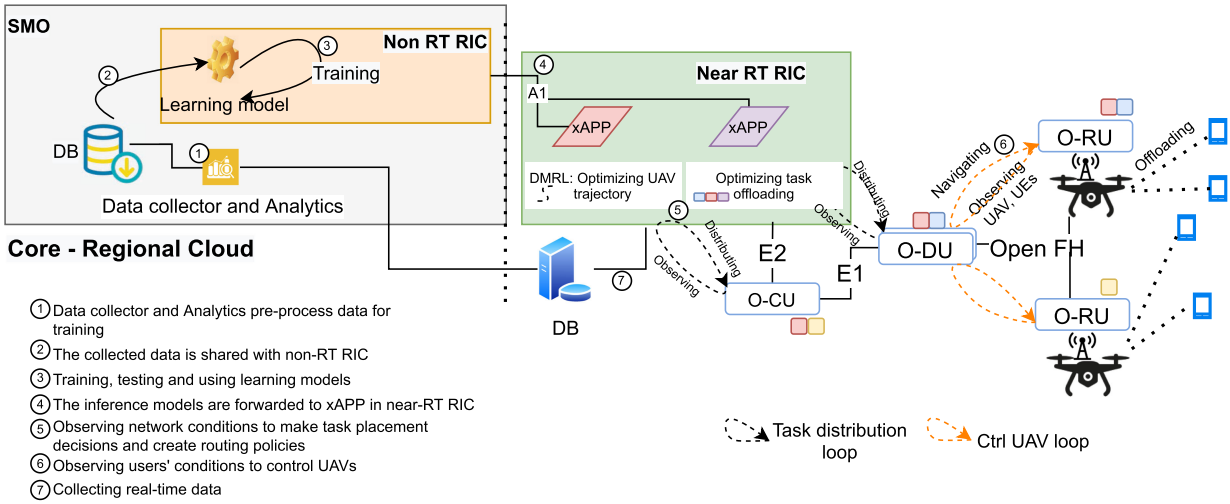



Fig. 2. Overview of the proposed architecture for U-ORAN.

$$\textbf{UTOT}: \max \quad \sum_{t=0}^{T-1} \sum_{i \in \mathcal{I}} u_i(\boldsymbol{x_i}, \boldsymbol{w_i}) \tag{6a}$$

$$\text{s.t.} \quad \sum_{n \in \mathcal{N}} x_{in}(t) = 1, \forall i \in \mathcal{I} \tag{6b}$$

$$\sum\nolimits_{i \in \mathcal{I}} \lambda_i c_i x_{in}(t) \leq \mu_n, \forall n \in \mathcal{N}, \tag{6c}$$

$$\sum\nolimits_{i \in \mathcal{I}} w^i_{(mn)}(t) \leq C_{mn}, \forall (mn) \in \mathcal{E}, \tag{6d}$$

$$\sum_{t=0}^{T-1} E_v^{\text{fly}}(t) + E_v^{\text{exc}}(t) \leq \bar{E}_v, \forall v \in \mathcal{V}, \tag{6e}$$

$$L_n^{\text{proc}} x_{in}(t) + L_i^{\text{trans}} \leq L_i, \forall i \in \mathcal{I}, \tag{6f}$$

where (8e) is the constraint to guarantee the safety level of the battery of UAV $v$ with the execution power consumption of UAV $v$ calculated by $E_v^{\text{exc}}(t) = \sum_{t=0}^{T-1} \sum_{i \in \mathcal{I}} \lambda_i p_i x_{in}(t)$. We denote $p_i$ as the amount of energy consumption to process 1-bit data.

In summary, UTOT is formed as an online optimization model that aims to maximize the total utility during the controlling period $T$ with the controlling variables including the association data rate $w^i_{(mn)}(t)$ of user $i$ on the link $(mn)$ and the decision offloading variable $x_{in}(t)$ to serve task $i$ at node $n$. UTOT is a mixed integer non-linear programming (MINLP) problem, of which the complexity to find the optimal solution (due to the complex constraints and variables) can be exponential. We note that the controlling UAV variables are implicitly presented via the data rate variable on the link between UAVs and users since at a coordinate $o_v(t)$, UAV $v$ makes an association decision to optimize the communication data rate $w^i_{(mn)}(t)$ to users. The movement of users affects the communication channel so that measuring the signal strength can help UAVs make appropriate decision to connect to ground users. In fact, the high complexity of joint UAV-trajectory and resource allocation problem has been discussed in many prior works [14], [15], [16], [20]. Therefore, finding an efficient mechanism for this problem is still interesting in research.

## 4 Proposed Methods

Due to the high complexity of UTOT, we propose a learning control loop to tackle this problem including two procedures: i) the UAV-based control procedure to optimize UAV trajectory and ii) the offloading task allocation procedure to distribute offloading tasks to appropriate execution containers. According to the RAN Intelligent Controller (RIC) introduced by O-RAN [9], the designed control loop for UTOT constitutes a two-fold manner. First, we employ a multi-agent reinforcement learning method to acquire an optimal UAV trajectory and optimize communication channels between users and UAVs to offload tasks. Second, we perform the resource allocation and routing based on the split O-RAN architecture. These control loops are online and interdependent. Specifically, at time $t$, UAVs are controlled to optimize the user's associations while the task allocation procedure makes a decision based on the current association between UAVs and users. The detail of the overview architecture and proposed methods is presented in the next sub-sections.

### 4.1 Overview Architecture

The architecture to solve UTOT and operate the system is proposed based on the general intelligent O-RAN architecture [30]. As shown in Fig. 2, the flow of the proposed architecture is comprised of six procedures from the beginning to the deployment steps. At first, data is collected and analyzed for training the learning models at the Non-RT RIC. These models are stored and delivered to xApps to be utilized in the Near-RT RIC after training and testing. Based on container images, they may be readily stored, published, and launched at various locations within the system. As previously stated, we run two learning models (UAV-based control and job allocation) on separate xApps. To communicate, manage, and train the learning model of UAVs, a centralized controller is used in this architecture. After training, each UAV can distributively operate and make decisions by themselves. The control loop continuously observes the environment to control UAVs and process offloading tasks. Simultaneously, another centralized optimization module in Near-RT RIC receives offloading requests from users to calculate and determine the optimal resource units to execute the offloading tasks. This procedure is operated under the following control loop to optimize the offloading task in the U-ORAN system.

**S1**: Deploying the number of UAVs based on the coverage area of UAVs.

Authorized licensed use limited to: National Yang Ming Chiao Tung University. Downloaded on October 19,2025 at 06:42:59 UTC from IEEE Xplore. Restrictions apply.

**S2**: At the time $t$, gathering SNR and offloading requests of all users.
**S3**: Based on the observation information (i.e., locations, SNRs, battery level of UAVs, $\lambda$), solving **P1** (by Algorithm 1 or Algorithm 2) to optimize the user-UAV association and UAV coordinates.
**S4**: Based on the association results in Step 3, solving **P2** (by Algorithm 3) to optimize the selected computing nodes and routing paths for serving offloading tasks.
**S5**: Going to Step 2.

In the next subsections, we present the proposed learning methods to solve UTOT. As shown in the previous section, this problem is not trivial to address due to mixed binary and real variables and non-convex constraints. Furthermore, the location, resource allocation and routing variables are coupled so that UTOT is more challenging to be solved. To address this problem, we decompose UTOT into two sub-problems. In the former, we find the best locations of UAVs to establish communication between UAVs and users. Based on this association, we solve the latter to optimize resource allocation and routing paths for offloading tasks.

## 4.2 UAV-Based Control Modeling With Stochastic Game

We first consider the UAV-based controlling sub-problem to make the association decision between UAVs and users as follows:

$$\textbf{P1}: \max \quad \sum_{t=0}^{T-1} \sum_{i \in \mathcal{I}} u_i(\boldsymbol{x_i}, \boldsymbol{w_i}) \tag{7}$$

$$\text{s.t.} \quad \sum\nolimits_{v \in \mathcal{V}} x_{iv}(t) = 1, \forall i \in \mathcal{I}, t = 0, \ldots, T-1, \tag{8}$$

$$\sum\nolimits_{i \in \mathcal{I}} \lambda_i c_i x_{iv}(t) \leq \mu_v, \forall v \in \mathcal{V}, t = 0, \ldots, T-1, \tag{9}$$

$$\sum\nolimits_{i \in \mathcal{I}} w_{iv}(t) \leq C_v, \forall v \in \mathcal{V}, \tag{10}$$

$$\sum\nolimits_{t=0}^{T-1} E_v^{\text{fly}}(t) + E_v^{\text{exc}}(t) \leq \bar{E}_v, \forall v \in \mathcal{V}, \tag{11}$$

where $C_v$ is the bandwidth capacity of UAV-BS $v$. Since the sub-problem **P1** only considers the communication between UAVs and users, the routing variable can be simply presented by $w_{iv}(t)$ as the uplink data rate from user $i$ to UAV $v$ that is determined mainly based on the distance between UAVs and users. In order to solve **P1**, we reformulate the UAV-based control model as a stochastic game and then propose a reinforcement learning algorithm to solve **P1**. We also assume these UAVs only communicate to users and there is no communication channel among UAVs.

We advocate the multi-agent reinforcement learning method which is based on the Markov decision process (MDP). All the notations related to the reinforcement learning method are presented in Table 2.

### 4.2.1 Background of Q-Learning Method

Typically, an MDP includes a tuple $(\mathcal{S}, \mathcal{A}, R, P)$ where $\mathcal{S}$ is a set of states; $\mathcal{A}$ is a set of actions of the agents, $R(s, a, s')_{s,s' \in \mathcal{S}, a \in \mathcal{A}}$ is a reward function to obtain reward for each state $s$, action $a$ and new state $s'$ and $P(s, a, s')$ is a transition function that presents the probability of changing from state $s \in \mathcal{S}$ to $s' \in \mathcal{S}$ by an action $a \in \mathcal{A}$.

TABLE 2
Notations

| | Notations of the RL-based model |
|---|---|
| $\mathcal{S}$ | The set of states indexed by $s$ and $s'$. |
| $\mathcal{A}$ | The set of actions indexed by $a$. |
| $\mathcal{S}_v$ | The set of states of UAV $v$. |
| $\mathcal{A}_v$ | The set of actions of UAV $v$. |
| $R(s, a, s')$ | The reward function for each state $s$, action $a$ and next state $s$ with value $r$ calculated by (18). |
| $P(s, a, s')$ | The probability function of a transition from $s$ to $s'$ with action $a$. |
| $\pi$ | The policy to go from a state to another state. |
| $\gamma$ | The discount factor. |
| $Q(s, a)$ | The Q-value of state $s$ and action $a$. |
| $Q(s, a\|\omega)$ | The approximation Q-value of state $s$ and action $a$ with the weight $\omega$. |
| $V_{\pi,v}(s)$ | The value of state $s$ of UAV $v$ following $\pi$. |
| $N_t(s, a)$ | The number of times the state $s$ selects action $a$ until $t$. |
| $\mathbb{E}(.)$ | The expectation function. |
| $\mathcal{L}(.)$ | The loss function. |

At each moment $t$ with a given state $s$, the agent decides a strategy $\pi(s)$ to select an action $a$ to maximize the expected value $\sum_{t=0}^{\infty} \gamma \mathbb{E}(r(t)|\pi)$, where $r(t)$ is the reward value at time $t$ and $\gamma \in [0, 1)$ is the discount factor. In Q-learning method, the optimal strategy $\pi^*$ exists with an optimal Q-value [31]

$$Q^*(s, a) = R(s, a, s') + \gamma \sum_{s' \in \mathcal{S}} P(s, a, s') \max\nolimits_{a' \in \mathcal{A}} Q^*(s', a').$$

In case of the non-deterministic scenario, the Q-update rule is modified by

$$Q^t(s, a) = (1 - \nu_t) Q^{t-1}(s, a) + \nu_t [r(s, a) + \max\nolimits_{a' \in \mathcal{A}} Q^{t-1}(s', a')], \tag{12}$$

where $\nu_t = \frac{1}{1 + N_t(s,a)}$ and $N_t(s, a)$ is the number of times that the state $s$ selects action $a$ until $t$.

### 4.2.2 Multi-UAV Reinforcement Learning

Based on the extended model of multi-agent RL [31], we formulate the stochastic multi-UAV control model as follows.

**Definition 1.** *A stochastic multi-UAV control model can be defined as a tuple $(\mathcal{V}, \mathcal{S}, \mathcal{A}, R, P)$, where $\mathcal{S}$ is the discrete set of environmental states that consists of the possible status of all agents, $\mathcal{A} = A_1 \times A_2 \times \ldots A_{|\mathcal{V}|}$ is the joint action set of all agents, $R : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \to \mathbb{R}$ is the reward function and $P : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \to [0, 1]$ is the transition function.*

For each agent, there is a strategy $\pi_v : \mathcal{S} \to \mathcal{A}_v$, where $\mathcal{A}_v$ is the action set of UAV $v$, respectively. Similar to the single-agent MDP, each agent aims to find a policy $\pi_v : \mathcal{S} \to \mathcal{A}_v, \forall v \in \mathcal{V}$. Suppose this policy is stationary, hence, we have

$$V_{\pi,v}(s) = \mathbb{E}_\pi[r_v(t)|s_0 = s], \forall s \in \mathcal{S} = \mathbb{E}_\pi \left[ \sum_{t=0}^{\infty} \gamma(t) r_v(t) | s_0 = s \right] \tag{13}$$

where $r_v(t)$ is the reward at time $t$ for the agent $v$. In addition, the Q-value for each state-action is calculated by

Authorized licensed use limited to: National Yang Ming Chiao Tung University. Downloaded on October 19,2025 at 06:42:59 UTC from IEEE Xplore. Restrictions apply.

$$Q_{\pi,v}(s,\{a_v\}_{v\in\mathcal{V}}) = R_v(s,a_v,s') + \gamma\sum_{s'} P(s,\{a_v\}_{v\in\mathcal{V}},s')V_{\pi,v}(s). \quad (14)$$

With a fully cooperative assumption, the Q-value adopted Bellman equation for multi-agent RL is given by

$$Q_v^*(s,a_v|\pi_{-v}) = \sum_{a_{-v}}\pi_{-v}(a_{-v},s)\Big[r(s,a_v,a_{-v}) + \gamma\sum_{s'}P(s'|s,a_v,a_{-v})\max_{a_v'}Q_v^*(s,a_v')\Big] \quad (15)$$

In contrast to the single-agent MDP, we aim to cooperate all UAVs to find the optimal policy $\pi_v^*$ for each UAV.

**Definition 2.** *A joint policy $\{\pi_v\}_{v\in\mathcal{V}}$ is a Nash equilibrium if each $\pi_v$ is the best response to other UAVs.*

The definition means that for each UAV $v$ with any other strategy $\pi_v'$, we have

$$Q^*(s,\{\pi_v(s)\}\cup\pi_{-v}) \geq Q^*(s,\pi_v'(s)\cup\pi_{-v}),$$

where $\pi_{-v}$ is the joint strategy excluding UAV $v$ and $\pi_v'(s)$ is any other policies.

Following the general Q-learning method with a multi-agent reinforcement learning model, we study the solution for **P1** in which each UAV is an agent controlled to optimize the objective of **P1**.

*Environment State.* At time $t$, agent $v$ observes a state $s_v(t)$ that can be given by $s_v(t) = \{o_v(t), o_i(t), w_{vi}(t), E_v(t)\}$. The observation state of a UAV determines whether its SNR level $\tau_{vi}(t)$ is enough for the transmission (i.e., greater than or equal to a required threshold epsilon $\epsilon$). Based on Shannon formula, we can convert SNR to data rate by $w_{iv}(t) = B\log_2(1+\tau_{vi}(t))$. Hence, if $\tau_{vi}(t) \leq \epsilon$ then $x_{vi}(t) = 0$ and $w_{vi}(t) = 0$. In practice, the location of users might not be necessary which can be replaced by SNR measurement.

*Action.* Based on the current policy $\pi_v$, at time $t$, agent $v$ observes a state $s_v(t)$ to select an action $a_v(t)$ comprised of a tuple $a_v(t) = (o_v(t), \{x_{v1}(t),\ldots,x_{vI}(t)\})$. It means that UAV $v$ decides to move to the new location and associate with a set of users. The action of each UAV is designed following the formulation of **P1** to make controlling decisions of UAVs and associations with users. We denote $A_{-v}(t)$ as the actions of other UAVs at time $t$, i.e., $A_{-v}(t) \in \mathcal{A}\setminus A_v$.

*Reward.* Since the reward leads each agent to its optimal policy, which determines the association and controlling decisions of a UAV, we design the reward function of UAV $v$ based on **P1** by combining the utility and the energy cost as follows

$$r_v(t) = \rho\sum\nolimits_{i\in\mathcal{I}_v} u(w_{vi}(t))x_{vi}(t) - \eta(E_v^{\text{fly}}(t) + E_v^{\text{exc}}(t)) + M, \quad (16)$$

where $M$ is a large positive constant, $\rho$ and $\eta$ are the coefficient parameters to convert these aspects to the same unit.

In the cooperative scenario, UAV $v$ at time $t$ and state $s_v$ has a mixed strategy $\pi_v(s_v) = \{\pi_v(s_v,a_v)|a_v \in A_v\}$. A joint strategy $\pi = \{\pi_v(s_v)\}_{v\in\mathcal{V}}$ is a vector strategy of all UAVs. Consider for a long-term reward of UAV $v$, we have

$$\mathbf{r}_v = \sum_{t=0}^{+\infty}\gamma r_v(t) = \mathbb{E}\left(\sum_{t=0}^{+\infty}\gamma r_v(t)|s_v(0)\right), \quad (17)$$

where $\mathbb{E}(.)$ is the expectation function. Since the UAVs have different expected rewards depending on the joint strategy of all UAVs, each cannot maximize its expected reward.

**Definition 3.** *If a joint strategy $\pi^* = \times\pi_1^*\times\pi_2^*\times\ldots\times\pi_V^*$ can maximize all the expected rewards of all UAVs, then this collection of strategies exists a Nash equilibrium (NE). It means that $\mathbf{r}_v(\pi_v^*) \geq \mathbf{r}_v(\pi_v'), \forall\pi_v'$.*

Hence, by cooperating with multiple agents, each UAV will aim at the best action to benefit other UAVs in a Nash equilibrium (NE)). Since we consider a system without a UAV-UAV communication, a joint strategy at a centralized controller is regarded as a task to find an NE strategy. Based on the cooperation between UAVs, we present the following solutions coordinated by a centralized controller.

*Full Cooperation Mechanism.* In order to fully cooperate, each UAV will send its observations and actions to the controller. With a joint state-set $\mathcal{S}$ and joint action set $\mathcal{A}$, the controller manages the Q-table. For the reward, at each time step all agents observe a joint reward value $r_1(t) = r_2(t) = \ldots = r_V(t)$.

According to [32], there are some simple solutions for obtaining the largest Q-value related to the Nash equilibrium. However, the centralized technique is difficult to employ since the cardinality of the action space is large, making Q-table management problematic. We use the projection technique provided in [18] in this paper in order to manage a smaller Q-table corresponding to the present observation of the agents. In Algorithm 1, the algorithm is described. At time $t$, a UAV observes its environment (Line 8) and updates its state at the controller (Line 9, 10). The joint action is executed to obtain the NE (Line 11) and updated to all agents (Line 12). The reward is updated to continue at the next iteration (Line 13).

However, besides the challenge to manage the joint state and action spaces, a cooperative approach has the challenge to obtain a convergence if each agent has a different reward function since the experience of each agent involves the experience policies of others.

*Asynchronous Cooperation Approach.* To cope with a large joint state and action space in a joint environment, an asynchronous cooperation approach to extend Algorithm 1 can be considered, in which an agent observes the environment and only sends its information to the controller if the change in its environment can trigger an update (e.g., the level of SNR satisfies the threshold, number of user requests). The controller will gather all of the observed information of the transmitting agents to make a coordinated action to obtain a Nash equilibrium. Hence, the algorithm for the asynchronous cooperation approach is not changed. Specifically, we add only a trigger based on SNR in the forwarding step (Line 10 of Algorithm 1) to update the agent's observation.

*Deep Q-Learning Approach.* Despite the projection and asynchronous improvements, managing a Q-table can still be a big challenge in a large-scale setting with a large coverage area and a large number of joint UAVs' actions. Following [33], we investigate a deep Q-network (DQN) approach

Authorized licensed use limited to: National Yang Ming Chiao Tung University. Downloaded on October 19,2025 at 06:42:59 UTC from IEEE Xplore. Restrictions apply.

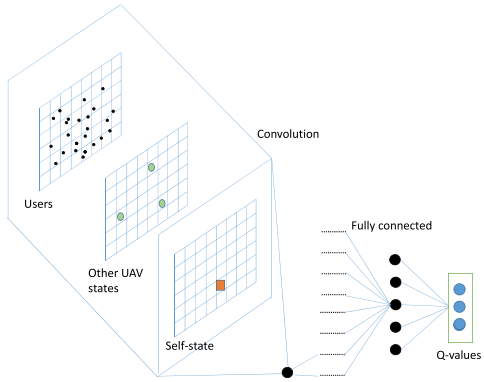



Fig. 3. The architecture of DQN.

in which the convolution neural network (CNN) [33] is used to represent the input state $s$ and the output of this neural network indicates all possible actions with the given input state $s$. In general, DQN can approximate the optimal policy in the Q-learning approach. In particular, each agent state is represented as an image snapshot to input the neural network. The outputs will estimate the Q-values of all agents.

Following the updating Q-values (16), the extended DQN approximates Q-values by $Q(s, a|\omega)$, where $\omega$ is the weight parameter of the neural network. In detail, this parameter is updated to minimize the loss function at iteration $k$

$$\mathcal{L}(s, a|\omega_k) = (r_v(s, a) + \gamma \max_a Q(s', a|\omega_i) - Q(s, a|\omega_k))^2.$$

As a typical method of updating weights, the value at iteration $k+1$ is as follows $\omega_{k+1} = \omega_k + \nu \nabla_{\omega_k} \mathcal{L}(\omega_k)$, where $\nu$ is the learning rate.

To apply DQN in our work, we take the advantage of the convolution neural network to encode the environment as shown in Fig. 3. We follow the typical DQN architecture and apply it to our problem. We use the image size $3 \times L \times L$ corresponding to 3 snapshot channels related to *user information* (i.e., user coordinate or connection level SNR of users with UAV $v$), states of all UAVs excluding $v$ and the current state of UAV $v$. The convolution neural network provides an efficient way to join the global information of the environment to be encoded as a pixel value. In fact, the approach is promising to allow us to extend the number of layers when the environment has more requirement information such as obstacles. The output layer of the network represents the Q-values for all the actions that can be taken by the UAV. This design allows us to solve **P1** in a distributed manner in which each UAV is controlled by a controller. However, this design model does not guarantee an optimal joint action since a selected action of an agent is based on the given information of other UAVs.

The most difficult aspect of this approach is the training. To avoid a direct influence from one agent on another and to ensure that all agents are incorporated, we train one agent at a time while maintaining the policies of all other agents. The policy acquired by a trained agent is shared with all others after a certain number of iterations. Specifically, all the steps of the DQN-based algorithm are presented in Algorithm 2. We use the replay memory to store transactions during the training period. At each episode, an agent will observe its environment to update its state value (Lines 9-10). These observations will be gathered at the controller to distribute to other agents (Lines 11-12). An action is selected based on the output value of the neural network based on the input information (Line 13). The transition of a UAV continues until the terminal state and is added into the memory. A mini-batch is sampled to calculate the loss and update the weighted parameters of each agent following the gradient descent (Lines 18-19).

*Discussion.* Summarily, according to the definition and proposed methods, the main differences of them are as follows:

- The Q-learning method is off-policy in which an action strategy and the evaluation strategy are not the same strategy. A Q-table with state rows and action columns is created and updated with rewards for each action. In Q-learning, the action strategy simply follows $\epsilon$-greedy policy to update Q-table. However, in terms of the fully cooperative approach, the computation is burdened by a large joint state and action space even in the synchronous cooperation approach.
- The DQN method uses convolution neural networks to approximate value functions. It can address the limitation from the Q-learning method with small and discrete state and action space [14], [31]. However, in the currently designed algorithm without a communication channel between UAVs, it does not guarantee a Nash equilibrium at the convergence.

**Algorithm 1.** UAV-Controlling Algorithm With Multi-Agent Q-Learning Approach

1 Initialization: Set $t = 0$;
2 **forall** $v \in \mathcal{V}$ **do**
3 Inittialize $Q_v^t$;
4 Initialize the action value $a_v(0)$, the strategy $\pi_v(0)$ and the state $s_v(0)$;
5 **end**
6 **while** $t \leq T - 1$ **do**
7 **forall** $v \in \mathcal{V}$ **do**
8 Observe the environment (level of SNR, battery);
9 Update state $s_v(t)$;
10 Forward observation $(s_v(t), A_v)$ to the controller where $\bar{A}_v$ is the projection actions corresponding to state $s_v$;
11 **end**
12 Select a joint action $\{a_v\}_{v \in \mathcal{A}}$ corresponding to the NE value;
13 Forward actions to all agents;
14 Update reward $r_v(t)$ by (19);
15 Update $t = t + 1$
16 **end**

### 4.3 Online Learning and Matching Approach for the Task Allocation Problem

In this section, we consider the second sub-problem decomposed from UTOT. Given the solution of **P1** including the current location of UAVs and the association between UAVs and users, we formulate the task allocation sub-problem as follows.

Authorized licensed use limited to: National Yang Ming Chiao Tung University. Downloaded on October 19,2025 at 06:42:59 UTC from IEEE Xplore. Restrictions apply.

$$\textbf{P2}: \max \quad \sum_{t=0}^{T-1} \sum_{i\in\mathcal{I}, n\in\mathcal{N}} \alpha P_{in}\lambda_i c_i x_{in}(t) + \beta \sum_{(mn)\in\mathcal{E}} w^i_{(mn)}(t),$$

$$\text{s.t.} \quad \sum_{n\in\mathcal{N}} x_{in}(t) = 1, \forall i \in \mathcal{I}, \tag{18}$$

$$\sum_{i\in\mathcal{I}} \lambda_i c_i x_{in}(t) \le \mu_v, \forall n \in \mathcal{N}, \tag{19}$$

$$\sum_{i\in\mathcal{I}} w^i_{(mn)}(t) \le C_{mn}, \forall (mn) \in \mathcal{E}, \tag{20}$$

$$L^{proc}_n x_{in}(t) + L^{trans}_i \le L_i - L^{uav}_i, \tag{21}$$

where $\mathcal{N} = \mathcal{D} \cup \mathcal{C} \cup \mathcal{R}$, $L^{uav}_i$ is the given latency obtained from **P1** and $t = 0, 1, \ldots, T-1$. To solve **P2**, we design the online learning and matching framework to find an optimal container/task matching.

**Algorithm 2.** UAV-Controlling Algorithm With Multi-Agent DQN Approach

```
Initialization: Set t = 0, weighted parameters θ, replay memory D, reset Q-value;
foreach each training episode e do
  Reset replay memory D;
  Reset state, t = 0, for each agent;
  while t < T do
    t = t + 1;
    foreach UAV v do
      if s_v(t) ≠ terminal then
        Measure the level of SNR;
        Update state s_v(t);
        Forward observation (s_v(t), a_v) to the controller;
        Gather observeration of other UAVs;
        Select the best action based on the output Q-value of the neural network;
        Get reward r_v(t + 1) and next state s_v(t + 1);
      end
    end
    Store transitions into D;
    Sample random minibatchs b of transitions from D;
    Update loss function by
    L(θ_v) = Σ_{k=1}^{b} (π^t_{-v}(a_{-v}, s) / π^t_{-v}(a_{-v}, s)) [(Q(s_v, a_v)
              −Q(s_v, a_v : θ_v))^2];
    Perform a gradient descent step to update θ;
  end
end
```

### 4.3.1 A Primer on the Offline and Online Matching Problem

According to the prior works, some special classes of resource allocation problems have been investigated, such as AdWords and matching problems [19], [34]. In this paper, we consider an online resource allocation problem with the following basic description. Suppose there are a set $\mathcal{I}$ of buyers and a set $\mathcal{J}$ of items that will be selected by the buyers. Given a preference matrix $P_{ij}, \forall i \in \mathcal{I}, \forall j \in \mathcal{J}$ that represents the preference of buyer $i$ on item $j$. In addition, the buyer $i$ is willing to pay $r_{ij}$ to buy the item $j$ with the total $b_j$ available object type $j$. The buyers will arrive one by one and therefore, their preferences depend on their arrivals. Finally, the operator will receive revenue based on the amount of fee that buyers purchase. Hence, from the operator's perspective, the objective is to maximize the total revenue.

Based on this idea, we first consider an offline allocation problem with the given preference matrix $P$. We denote the assignment variable by $x_{ij}$ to indicate that buyer $i$ buys item $j$ or not. Hence, the simple offline matching problem is presented as follows.

$$\max \quad \sum\nolimits_{i\in\mathcal{I}} \sum\nolimits_{j\in\mathcal{J}} r_j P_{ij} x_{ij} \tag{22a}$$

$$\text{s.t.} \quad \sum\nolimits_{i\in\mathcal{I}} x_{ij} = 1, \forall j \in \mathcal{J}, \tag{22b}$$

$$\sum\nolimits_{i\in\mathcal{I}} P_{ij} x_{ij} \le b_j, \forall j \in \mathcal{J}, \tag{22c}$$

$$x_{ij} = \{0, 1\} \tag{22d}$$

To solve this integer problem, we relax the binary variable $x_{ij}$ to take its value in the range of [0, 1] and ignore the rounding issues [35]. In addition, we follow the method in [36] to add the regularization term $H(a) = -\sum_{j=1}^{n} a_j \log a_j$ in the objective function to convert this problem into a convex problem. Hence, we derive the partial Lagrangian function [37] by introducing the dual variable $\boldsymbol{\delta}$ and weighted parameter $\varphi$ as follows:

$$L = \sum_{i\in\mathcal{I}, j\in\mathcal{J}} (r_j P_{ij} x_{ij} \underbrace{-\varphi x_{ij} \log x_{ij}}_{\text{Regularization}}) + \sum_{j\in\mathcal{J}} \delta_j (\sum_{i\in\mathcal{I}} P_{ij} x_{ij} - b_j).$$

Based on the Karush-Kuhn-Tucker (KKT) conditions [37], we can derive the optimal solution based on the dual variable $\delta$ as

$$x^*_{ij} = \frac{1}{\Delta_j} \exp \frac{(r_{ij} - \delta_j) P_{ij}}{\varphi},$$

where $\Delta_j = \sum_{i\in\mathcal{I}} \exp \frac{(r_{ij} - \delta_j) P_{ij}}{\varphi}, \forall j \in \mathcal{J}$.

Consider an online problem in which a buyer arrives at time $t$ and selects his item $j$ and the preference list $P_{ij}$ is unknown because the item set is changed based on the prior selections. In general, this problem can be formed as an online learning model where we aim to minimize the following regret during a time horizon $T$.

$$\text{Regret}_T(x^*) = \sum\nolimits_{t=0}^{T-1} f_t(x_t) - f_t(x^*_t) \tag{23}$$

in which $f(x^*_t)$ is the optimal solution of (24) after obtaining the information of the arrivals and the preference. In other words, we can consider the offline problem as the benchmark for the online method. Therefore, the regret is used as a metric for adjusting the system.

### 4.3.2 Online Learning and Matching Approach for the Task Allocation Problem

Consider the sub-problem **P1**, it can be seen as an extension online matching problem in which we identify offloading tasks with buyers and the set of containers with the set of items and add more allocation constraints. The constraint capacity in (24c) is similar to the computing capacity of each container to serve offloading tasks.

Note that we simplify **P2** by relaxing the routing constraints (22) and (23) in order to find the best container and

Authorized licensed use limited to: National Yang Ming Chiao Tung University. Downloaded on October 19,2025 at 06:42:59 UTC from IEEE Xplore. Restrictions apply.

then we exploit the satisfied shortest paths to route the user requests.

*Online Greedy-Value Approach.* In this scenario, there is a simple strategy that each offloading task at time $t$ tries to optimize its benefit by solving its optimization problem

$$\max \quad \sum\nolimits_{n\in\mathcal{N}} u(x_{in}) \tag{24a}$$

$$\text{s.t.} \quad \sum\nolimits_{n\in\mathcal{N}} x_{in} = 1, \forall i \in \mathcal{I}, \tag{24b}$$

$$\sum\nolimits_{i\in\mathcal{I}} \delta_i c_i x_{in} \leq \mu_n, \forall n \in \mathcal{N}. \tag{24c}$$

At the arrival time $t$, the preference value $P_{in}$ can be obtained by calculating the available resource of each container $n$ to serve task $i$. This online greedy approach is simple but cannot optimize the objective in a long term since it only exploits the best strategy for the current time slot.

**Algorithm 3.** Online Learning and Matching Algorithm (OLMA) for **P2**

1 Initialization: type of tasks $m$, number of containers $n$, initial preference matrix **P**;
2 **for** $t = 1, \ldots, T$ **do**
3 Observe the type of arrival task $i$;
4 Calculate UCB based on (27);
5 **if** *$P(t)$ is changed* **then**
6 Assign task $i$ to the container that has a maximum UCB;
7 **end**
8 **else**
9 Solve (26) to assign task based on $P(t-1)$;
10 **end**
11 Update $P_{in}(t) = R_{in}(t)/N_{in}(t)$;
12 Project the solution to constraint (22) and (23);
13 **if** *(22) and (23) are sastified* **then**
14 Go to **Step 2**;
15 **end**
16 **else**
17 Remove the selected container out of the input;
18 Go to **Step 4**;
19 **end**
20 **end**

*Exploring Upper Confidence Bound (UCB) and Greedy-Value Approach.* This approach considers a strategy based on the combination between the UCB [31] and greedy-value approaches to make a matching decision. Let **D** denote a binary reward matrix, where each entry $D_{in}$ indicates whether or not the task $i$ is served by container $n$. Once the task arrival at time $t$, we let $N_{in}(t)$ denote the number of selections of the task $i$ that selects container $n$ and $R_{in}(t)$ be the amount of rewards we have by assigning task $i$ referred to the objective function (7). Hence, the average reward is calculated by $\bar{R}_{in}(t) = R_{in}(t)/N_{in}(t)$. Based on the UCB algorithm in the reinforcement learning method, the UCB function is presented by

$$UCB_{in}(t-1) = \begin{cases} \infty & N_{in}(t-1) = 0, \\ \bar{R}_{in}(t-1) + \sqrt{\frac{3\log(t)}{2N_{in}(t-1)}} & \text{otherwise.} \end{cases} \tag{25}$$

and the reward is updated by $R_{in}(t) = R_{in}(t-1) + D_{in}$.

Based on that, we propose an online learning and matching algorithm for the offloading task allocation presented in Algorithm 3. The idea of this algorithm is to combine UCB with the greedy method to enhance the objective value based on the learning knowledge. In further detail, when a job is received, the controller will categorize the task type in order to make an appropriate choice based on the type of containers (e.g., O-RU, O-DU, O-CU) (Line 3). If the preference is changed, the task will be delivered to the container that has the highest UCB value (Lines 5-7). Otherwise, we do the online matching by solving (26) with the previous **P** to obtain an assignment (Line 9). The preference matrix **P** is reinforced by updating in Line 11. The assignment will be evaluated by projecting on routing constraints (22) and (23) (Line 12). If it violates these constraints, these containers will be removed out of container lists to recalculate again from *Step* 4. Otherwise, offloading task $i$ will be assigned to the selected container and update the routing to continue processing the next arrival tasks.

**Theorem 1.** *The average regret of Algorithm 3 converges to 0 as $T \to \infty$ in the case of updating the dual variable based on the gradient descent.*

**Proof.** Based on the gradient descent method and the Lagrangian function, the dual update is as follows:

$$\delta_{t+1} = \delta_t - \rho_t \nabla f_t(\delta_t, P_t).$$

Hence, we have

$$\begin{aligned} ||\delta_{t+1} - \delta^*||^2 &= ||\delta_t - \rho_t \nabla f_t(\delta_t, P_t) - \delta^*||^2 \\ &= ||\delta_t - \delta^*||^2 + (\rho_t \nabla f_t(\delta_t, P_t))^2 - 2\rho_t \nabla f_t(\delta_t, P_t)(\delta_t - \delta^*) \end{aligned} \tag{26}$$

Then, we can derive

$$\begin{aligned} \nabla f_t(\delta_t, P_t)(\delta_t - \delta^*) =& \frac{1}{2\rho_t}(||\delta_t - \delta^*||^2 - ||\delta_{t+1} - \delta^*||^2) \\ &+ \left(\frac{\rho_t}{2}\nabla f_t(\delta_t, P_t)\right)^2. \end{aligned} \tag{27}$$

Consider the regret at time $t$

$$\begin{aligned} f_t(\delta_t, P_t) - f_t(\delta^*, P^*) \leq& |f_t(\delta_t, P_t) - f_t(\delta_t, P^*)| \\ &+ |f_t(\delta_t, P^*) - f_t(\delta^*, P^*)|. \end{aligned} \tag{28}$$

Based on the convexity of the objective function $f(.)$, we have

$$f_t(\delta_t, P^*) - f_t(\delta^*, P^*) \leq \nabla f_t(\delta_t, P^*)(\delta_t - \delta^*). \tag{29}$$

Hence, if we fix $\rho_t = \rho$ we have (for sake of notations, we consider the time slot in range of $1 \leq t \leq T$)

$$\text{Regret}_T = \sum_{t=1}^{T} |f_t(\delta_t, P_t) - f_t(\delta_t, P^*)| + |f_t(\delta_t, P^*) - f_t(\delta^*, P^*)|$$

Authorized licensed use limited to: National Yang Ming Chiao Tung University. Downloaded on October 19,2025 at 06:42:59 UTC from IEEE Xplore. Restrictions apply.

TABLE 3
Simulation settings

| Parameters | Values |
|---|---|
| Simulation area | 500 m×500 m |
| Number of UAVs | [4,10] |
| Transmitting power of UAVs | [10, 16] dBm |
| Power of background noise | -96 d Bm |
| Terrain parameters | $\psi_1 = 11.95$ and $\psi_2 = 0.136$ |
| Container capacity ($P_v$) | [8, 16] core units |
| Cloud node capacity ($P_v$) | 100 core units |
| Service rate of a container ($\mu_v$) | [12, 24] Mpps |
| Propagation latency of a link | [10, 30] ms |
| The average additional loss of LOS ($\eta_{LoS}$) | 2 |
| The average additional loss of NLOS ($\eta_{NLoS}$) | 21 |
| Total energy of UAVs | [3000 and 8000] mAh |
| Latency threshold setting ($\bar{L}$) | [30, 150] ms |

$$\leq \sum_{t=1}^{T} |f_t(\delta_t, P_t) - f_t(\delta_t, P^*)| + \nabla f_t(\delta_t, P^*)(\delta_t - \delta^*)$$
$$\leq \sum_{t=1}^{T} |f_t(\delta_t, P_t) - f_t(\delta_t, P^*)| + \frac{1}{2\rho}(||\delta_0 - \delta^*||^2 + \sum_{t=1}^{T} \frac{\rho}{2}(\nabla f_t(\delta_t, P_t))^2. \quad (30)$$

Given an upper bound of the dual variable $||\delta_0 - \delta^*||^2 \leq \xi^2$ and $\nabla f_t(\delta_t, P_t))^2 \leq \chi^2$, we can have

$$\frac{1}{2\rho}(||\delta_0 - \delta^*||^2 + \sum_{t=1}^{T}\left(\frac{\rho}{2}\nabla f_t(\delta_t, P_t))^2\right) \leq \frac{1}{2\rho}\xi^2 + \frac{\rho}{2}\chi^2 = \xi\chi\sqrt{T},$$

where we set $\rho = \frac{\xi}{\chi\sqrt{T}}$. According to [38], if we apply the UCB strategy for every time slot $t$ from 1 to $T$, the total regret after $T$ is $\mathcal{O}(T \log T)$. It means that with a positive $\epsilon$, we have

$$\sum_{t=1}^{T} |f_t(\delta_t, P_t) - f_t(\delta_t, P^*)| \leq \epsilon\sqrt{T \log T}, \quad (31)$$

Thus, based on the L'Hopital's rule, we can have

$$\lim_{T\to\infty} \frac{\text{Regret}_T}{T} \leq \lim_{T\to\infty} \epsilon\sqrt{\frac{\log T}{T}} + \xi\chi\sqrt{\frac{1}{T}} = 0.$$

□

## 5 Numerical Results

In order to evaluate our proposed algorithms, we first discuss our network setting and then present the extensive simulation results of our work.

### 5.1 Network Settings

We create a simulation environment by using Python and the Keras framework to develop the distributed and flying U-ORAN network and train the system performed on an Intel Xeon(R) computer (CPU E5-2670 2.60 GHz × 4, 16 GB Memory). Specifically, we generate a network with a set of UAV base stations to serve terrestrial users. The configuration parameters used in our simulations are listed in Table 3.

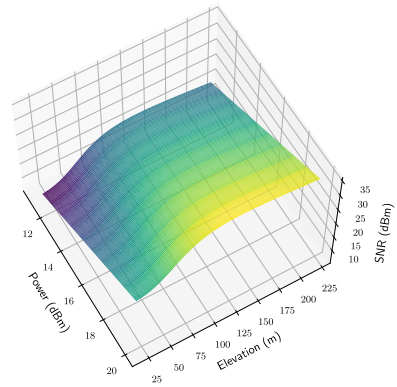



Fig. 4. The optimal elevation and power settings of UAVs.

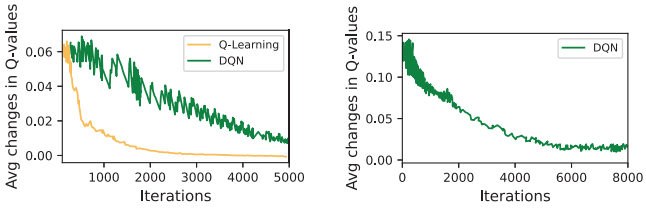
We randomly generate 400 ground users, each user performs a random walk algorithm in an area of 500 m × 500 m. To simplify our model, we conduct an optimal setting for the elevation of UAVs. As shown in Fig. 4, we consider a fixed elevation of UAV which is set by an optimal height range of [100 m, 125 m] corresponding to our current setting. The speed of UAVs is set by 5 m/s while the duration for each time slot to adjust UAVs is 1s.

For the offloading tasks, we randomly create tasks based on generated ground users and the latency threshold of each task is set from 30 ms to 150 ms. The size of requested data of the offloading tasks is set in the range of [50 to 100] $\times 10^4$ bits and the required cycles of the processors are in the range of [5, 20] cycles. Environment settings to calculate the path loss in our simulation are static as shown in Table 3.

To evaluate the performance of our proposed methods, we make the comparison between the following methods:

- Q-learning: We name Algorithm 1 as Q-learning to present in the simulation result. In order to evaluate the performance of Algorithm 1, we create a *small-scale setting* with the same area and a maximum of 70 ground users.
- DQN: We name Algorithm 2 as DQN which is used in all of the evaluations. Different from Q-learning, we evaluate this method in both *small-scale* and *large-scale* settings (with a maximum of 400 users and the number of UAVs that can scale up to 10).
- K-Mean [3]: In this method, given the coordinates of users, we group the number of users based on K-Mean method and calculate the cluster center locations at each time slot that are used to navigate UAVs. In fact, obtaining all location information of users is not practical but provides a baseline result to conduct the performance.
- Fixed base stations (Fixed): In this method, we divide the map into multiple regions corresponding to the number of UAVs. We fix the location of each UAV in its region which plays as a tower to serve offloading tasks.
- Multi-region UAV deployment [15]: Similar to the fixed baseline, we apply the solution of [15] to control a UAV in its region and make offloading decisions. Since the

Authorized licensed use limited to: National Yang Ming Chiao Tung University. Downloaded on October 19,2025 at 06:42:59 UTC from IEEE Xplore. Restrictions apply.




(a) Small-scale setting. (b) Large-scale setting.

Fig. 5. Changes of Q-values.

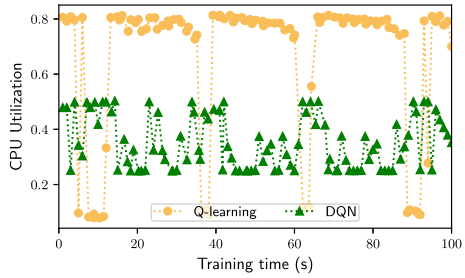



Fig. 6. CPU utilization during the training phase.

formulated model is different from ours, we can only apply the idea of [15] to our model.

- Greedy: This baseline is the greedy strategy presented in Section 4.3.2 where the offloading task is distributed to the best container to maximize the objective of **P2**.
- OLMA: This is the online learning and matching algorithm presented in Algorithm 3.
- Exhaustive: We consider this baseline as the offline solution in which we use the brute force method to search for an optimal solution of **P2**. However, due to the high complexity of this method, we limit the window time to 20 time slots. Note that all the generated tasks are verified by this solution to ensure that each has at least one allocation strategy.

## 5.2 Results

### 5.2.1 Convergence

We first evaluate the convergence of our proposed algorithms to evaluate the performance in finding the optimal policies. The comparison between the multi-agent Q-learning and DQN approaches is made by using the small-scale setting of 4 UAVs with all the states and actions that are discretized based on [39]. In such a setting, we are able to handle the Q-table of joint actions of all UAVs. Fig. 5 shows the change of Q-values in which the Q-learning method converges faster than DQN after 2,000 iterations. The reason is that all the UAVs use the same reward function and the actions are in the same direction which is selected to obtain the highest rewards.

However, the overhead of using the Q-learning method is higher than DQN in terms of CPU utilization as demonstrated in Fig. 6. During 100s of the training phase with our simulation environment (Intel Xeon(R) CPU E5-2670 0 2.60 GHz $\times$ 4, 16 GB Memory), the Q-learning method utilizes up to 80% of CPU in most of the time slots to calculate the optimal Q-value of joint actions. Compared to Q-learning, the CPU utilization of DQN is smaller and more stable in the range of [0.2, 0.5].

Fig. 7b illustrates a large-scale setting in which DQN converges after the 8000th iteration. In a large-scale setting, we can only perform the evaluation by DQN as shown Fig. 5b. DQN gradually converges after 8,000 iterations in the training. Compared to the large setting, the change value in the Q-table of the Q-learning method is smaller than that of DQN.

In addition, we illustrate the average reward of these approaches in the training phase. In Fig. 7, we present the average reward of Q-learning and DQN from the beginning to the end of training. By jointly selecting a Nash equilibrium action at each iteration, the rewards of UAVs have a small gap (Fig. 7a) compared to the reward gap in DQN. Each UAV uses the shared information of others but it does not consider the action of others so that its average reward is different from others (Fig. 7b). On average, the return value of DQN (with 22.3) is smaller than that of Q-learning (with 26.8).

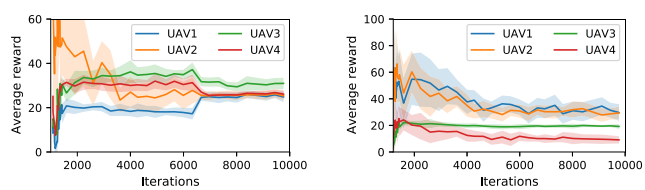
### 5.2.2 Evaluation of Controlling UAV-BSs

In this subsection, we evaluate the performance of our algorithms by taking into account the following evaluation aspects.

*Trajectory of UAVs.* Fig. 8 makes a comparison between different methods in terms of controlling the trajectory of UAVS.

First, we compare the movement of a UAV by using different methods, shown in Fig. 8a. Q-learning and DQN have similar trajectories during 100 flying time slots. The trajectory is captured from UAV1, which is presented with other UAVs in Fig. 8b. The figure displays that K-Mean baseline based on the centralized cluster location to control UAVs has a longer trajectory than that of Q-learning and DQN. Even though this method can obtain a minimum UAV-user distance to have the best communication, it results in a long trajectory of the UAV. In practice, having all the user location information to navigate UAVs is difficult and unnecessary. However, it is considered a good baseline to evaluate the controlling UAV of our proposed method. Based on Fig. 8a, the result demonstrates that Q-learning and DQN have good performance to automatically control UAVs for serving ground users.

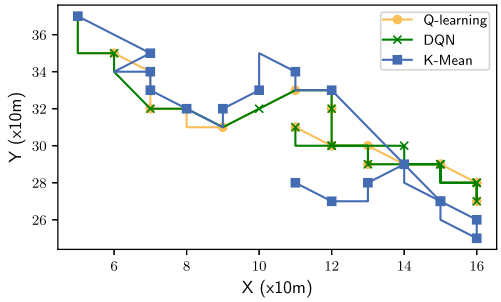
In addition, Fig. 8b captures the trajectory of all UAVs using the reinforcement learning method (e.g, DQN) in which the shading areas represent the areas of user locations. The figure illustrates the movement of all UAVs during the simulation period to serve the offloading tasks. Combined with the shading area of ground users, we can observe that the trajectory of UAVs using DQN can cover most of the area on the map. In Fig. 8c, we plot the fixed



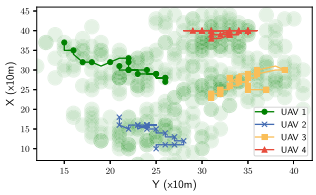

(a) Q-learning. (b) DQN.

Fig. 7. Average reward.

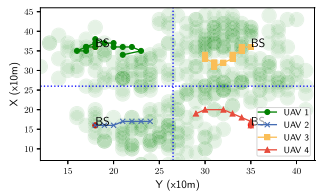
Authorized licensed use limited to: National Yang Ming Chiao Tung University. Downloaded on October 19,2025 at 06:42:59 UTC from IEEE Xplore. Restrictions apply.




(a) Trajectory comparison.




(b) Trajectory of UAVs.




(c) Trajectory of UAVs by the multi-region deployment.

Fig. 8. UAV trajectory.

locations of BSs used by the fixed baseline and the trajectory of UAVs used by the multi-region deployment baseline. Since each UAV operates in its divided region, the users who move close to the border are not served well compared to our proposed method. In order to illustrate better the result, we present the coverage rate in the next evaluation.

*Coverage Rate.* We make a comparison between our RL-based approaches and the baselines (e.g., K-Mean, Fixed, and Multi-region) demonstrated in Fig. 9. Due to the high complexity of Q-learning, we can only implement this algorithm with 4 UAVs. Fixed baseline has the lowest coverage rate compared to others, but it shows a lower bound to conduct the performance of others. Compared to K-Mean with an 87.6% coverage rate, both Q-learning and DQN have promising coverage rates of 70.2% and 83.2% on average, respectively. The multi-region baseline has a small gap compared to DQN. In fact, by isolating the map to multiple regions, we recognize that a UAV falls to a dilemma control when many users move to borders in different directions since the UAV does not have information of other UAVs. It shows the good performance of RL-based methods to control UAVs even though they do not need to provide the ground user location information.

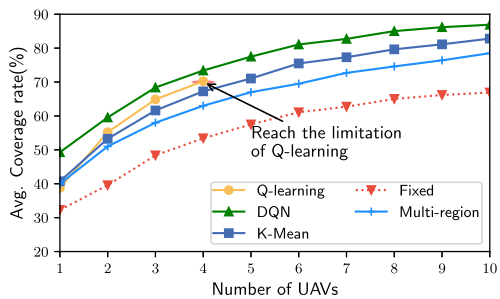
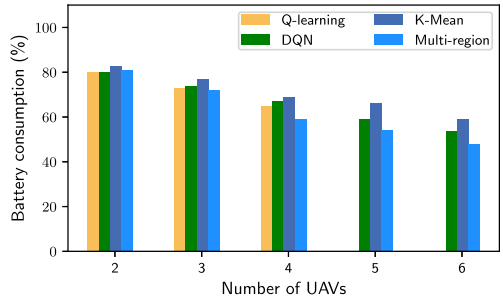
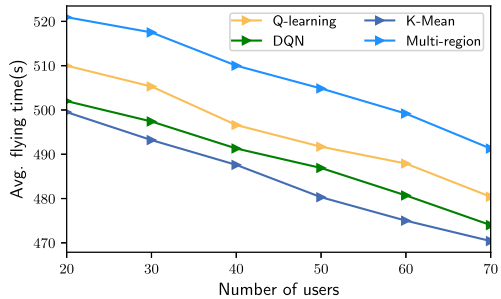



Fig. 9. Evaluation of the coverage rate.




(a) Energy consumption.




(b) Flying time.

Fig. 10. Evaluation of the energy consumption and flying time.

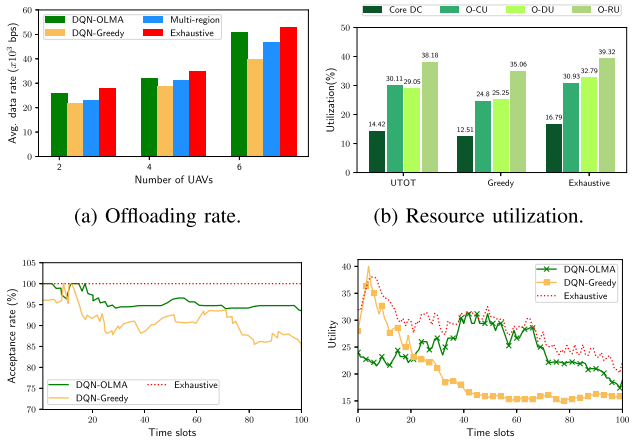
*Energy Consumption and Flying Time.* Finally, we evaluate the energy consumption and flying time which are derived by (1) and (8e). We evaluate the battery consumption in Fig. 10a utilizing different approaches during 100 time slots. K-Mean requires UAVs to track the user's locations, which consumes a lot of battery. The difference between Q-learning and DQN is small, indeed DQN uses more battery than Q-learning. For the multi-region, because UAVs only fly in a smaller area, this baseline uses less battery than others. Furthermore, this baseline has a lower coverage rate than others, resulting in fewer offloading tasks served by UAVs. As a result, UAVs in this method require less power consumption.

Related to the battery consumption, we evaluate the flying time of these methods by scaling the number of users in the system as shown in Fig. 10b. On average, the flying time of all methods decreases when scaling up the system. For K-Mean, the movement of UAVs follows the movement of users. Therefore, the average flying time of UAVs in this method is the lowest compared to Q-learning and DQN. The multi-region baseline demonstrates the longest flying time compared to others. Such a result can be explained based on the previous results (i.e., trajectory and energy consumption). This baseline has a big gap compared to Q-learning and DQN in the flying time. Using Q-learning to coordinate all UAVs achieves a better flying time compared to DQN. However, the gap between Q-learning and DQN is small in all settings.

### 5.2.3 Evaluation of Task Allocation

In this part, we consider the resource utilization and total utility in the system to evaluate the performance of the proposed algorithm.

Authorized licensed use limited to: National Yang Ming Chiao Tung University. Downloaded on October 19,2025 at 06:42:59 UTC from IEEE Xplore. Restrictions apply.




(a) Offloading rate. (b) Resource utilization.

(c) Acceptance rate. (d) Utility.

Fig. 11. Evaluation of task allocation.

*Offloading Data Rate.* Given the association from **P1** by using DQN (named DQN-OLMA), we compare the offloading rate with other baselines by scaling the number of base stations. By using more UAV-BSs, the average offloading rate of our method increases significantly from $26 \times 10^3$ bps to $51 \times 10^3$ bps corresponding to 2 UAVs and 6 UAVs. Such a result has a small gap compared to the Exhaustive baseline and outperforms DQN-Greedy (the combination between DQN and Greedy methods) and the Multi-region baseline. The DQN-Greedy has the lowest performance due to the resource allocation strategy (we will explain in the next). Compared to the Multi-region baseline, our proposed method has a higher data rate which is related to the better coverage rate and the resource allocation strategy.

*Resource Utilization.* In Fig. 11b, we show the system's resource utilization of different approaches. The resource usage of O-RU and O-DU containers is higher than that of O-CU and core DCs due to their compact configurations. Compared to DQN-Greedy baselines, DQN-OLMA consumes more resources, which is explained by its higher acceptance rate, as seen in Fig. 11c. DQN-OLMA has a 96% acceptance rate on average while DQN-Greedy only has a 91% acceptance rate. The resource utilization gap between DQN-OLMA and Exhaustive baseline is not significant in all types of containers (O-RU, O-DU, O-CU, and Core DC). The acceptance rate of the Greedy baseline is unstable and lowest and therefore, its resource utilization is the worst in this evaluation.

*Utility Evaluation.* Finally, we compare the utility values of our method with the Greedy and Exhaustive baselines in Fig. 11d. The Greedy obtains the highest utility in the early time slots but substantially decreases after that. Because this strategy uses the best containers greedily, they get fully utilized and can then accept more jobs. The Exhaustive baseline has a big gap compared to our method at first, but DQN-OLMA eventually gains the utility and reduces the gap to the Exhaustive result after 30-time slots. DQN-OLMA achieves a utility of 24.5 on average, which is greater than the Greedy baselines (19.3).

## 6 Conclusion

In this work, we studied the potential of the flying O-RAN architecture for serving offloading tasks. The proposed model and architecture can be considered as a promising solution to deploy flying base stations in the network and serve offloading tasks under the O-RAN architecture. We modeled the joint multi-UAV trajectory and offloading task allocation based on this architecture which aims to optimize resource allocation. We also showed a learning approach combining reinforcement learning and online learning methods is promising in solving UTOT. We then performed intensive numerical simulations with different settings and aspects to evaluate the performance of the proposed methods. Compared to baselines, our proposed algorithms achieve remarkable results in terms of resource utilization, utility values, and service acceptance rate.

## References

[1] B. Li, Z. Fei, and Y. Zhang, "UAV communications for 5G and beyond: Recent advances and future trends," *IEEE Internet Things J.*, vol. 6, no. 2, pp. 2241–2263, Apr. 2019.

[2] K. P. Valavanis and G. J. Vachtsevanos, *Handbook of Unmanned Aerial Vehicles*, Berlin, Germany: Springer Publishing Company, Incorporated, 2014.

[3] M. Mozaffari, W. Saad, M. Bennis, Y.-H. Nam, and M. Debbah, "A tutorial on UAVs for wireless networks: Applications, challenges, and open problems," *IEEE Commun. Surveys Tuts.*, vol. 21, no. 3, pp. 2334–2360, Third Quarter 2019.

[4] Drone tech planet, Feb. 2021. [Online]. Available: https://www.dronetechplanet.com/how-much-weight-can-a-drone-carry/

[5] Drone volt, Feb. 2021. [Online]. Available: https://www.dronevolt.com/en/expert-solutions/hercules-20/#features

[6] M.-K. Yoon, B. Liu, N. Hovakimyan, and L. Sha, "VirtualDrone: Virtual sensing, actuation, and communication for attack-resilient unmanned aerial systems," in *Proc. IEEE/ACM 8th Int. Conf. Cyber-Physical Syst.*, 2017, pp. 143–154.

[7] Y. Yazid, I. Ez-Zazi, A. Guerrero-González, A. El Oualkadi, and M. Arioua, "UAV-enabled mobile edge-computing for IoT based on AI: A comprehensive review," *Drones*, vol. 5, no. 4, 2021, Art. no. 148.

[8] O. Bekkouche, K. Samdanis, M. Bagaa, and T. Taleb, "A service-based architecture for enabling UAV enhanced network services," *IEEE Netw.*, vol. 34, no. 4, pp. 328–335, Jul./Aug. 2020.

[9] O-RAN, Jun. 2021. [Online]. Available: https://www.o-ran.org/resources

[10] H. Wang, H. Zhao, W. Wu, J. Xiong, D. Ma, and J. Wei, "Deployment algorithms of flying base stations: 5G and beyond with UAVs," *IEEE Internet Things J.*, vol. 6, no. 6, pp. 10 009–10 027, Dec. 2019.

[11] E. Kalantari, M. Z. Shakir, H. Yanikomeroglu, and A. Yongacoglu, "Backhaul-aware robust 3D drone placement in 5G+ wireless networks," in *Proc. IEEE Int. Conf. Commun. Workshops*, 2017, pp. 109–114.

[12] C. Qiu, Z. Wei, Z. Feng, and P. Zhang, "Joint resource allocation, placement and user association of multiple UAV-mounted base stations with in-band wireless backhaul," *IEEE Wireless Commun. Lett.*, vol. 8, no. 6, pp. 1575–1578, Dec. 2019.

[13] S. A. Huda and S. Moh, "Survey on computation offloading in UAV-enabled mobile edge computing," *J. Netw. Comput. Appl.*, vol. 201, 2022, Art. no. 103341. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1084804522000108

[14] B. Liu, H. Huang, S. Guo, W. Chen, and Z. Zheng, "Joint computation offloading and routing optimization for UAV-edge-cloud computing environments," in *Proc. IEEE SmartWorld, Ubiquitous Intell. Comput., Adv. Trusted Comput., Scalable Comput. Commun., Cloud Big Data Comput., Internet People Smart City Innov.*, 2018, pp. 1745–1752.

[15] J. Xiong, H. Guo, and J. Liu, "Task offloading in UAV-aided edge computing: Bit allocation and trajectory optimization," *IEEE Commun. Lett.*, vol. 23, no. 3, pp. 538–541, Mar. 2019.

[16] Z. Zhou *et al.*, "When mobile crowd sensing meets UAV: Energy-efficient task assignment and route planning," *IEEE Trans. Commun.*, vol. 66, no. 11, pp. 5526–5538, Nov. 2018.

[17] M. Li, N. Cheng, J. Gao, Y. Wang, L. Zhao, and X. Shen, "Energy-efficient UAV-assisted mobile edge computing: Resource allocation and trajectory optimization," *IEEE Trans. Veh. Technol.*, vol. 69, no. 3, pp. 3424–3438, Mar. 2020.

Authorized licensed use limited to: National Yang Ming Chiao Tung University. Downloaded on October 19,2025 at 06:42:59 UTC from IEEE Xplore. Restrictions apply.

[18] M. Lauer and M. Riedmiller, "An algorithm for distributed reinforcement learning in cooperative multi-agent systems," in *Proc. 17th Int. Conf. Mach. Learn.*, 2000, pp. 535–542.
[19] B. Kalyanasundaram and K. R. Pruhs, "An optimal deterministic algorithm for online matching," *Theor. Comput. Sci.*, vol. 233, no. 1/2, pp. 319–325, Feb. 2000.
[20] M.-A. Messous, S.-M. Senouci, H. Sedjelmaci, and S. Cherkaoui, "A game theory based efficient computation offloading in an UAV network," *IEEE Trans. Veh. Technol.*, vol. 68, no. 5, pp. 4964–4974, May 2019.
[21] G. Fragkos, N. Kemp, E. E. Tsiropoulou, and S. Papavassiliou, "Artificial intelligence empowered UAVs data offloading in mobile edge computing," in *Proc. IEEE Int. Conf. Commun.*, 2020, pp. 1–7.
[22] H. Mei, K. Wang, and K. Yang, "Joint cache content placement and task offloading in C-RAN enabled by multi-layer MEC," *Sensors*, vol. 18, no. 6, 2018, Art. no. 1826.
[23] X. Foukas and B. Radunovic, "Concordia: Teaching the 5G vRAN to share compute," in *Proc. ACM SIGCOMM Conf.*, 2021, pp. 580–596.
[24] G. O. Pérez, A. Ebrahimzadeh, M. Maier, J. A. Hernández, D. L. López, and M. F. Veiga, "Decentralized coordination of converged tactile internet and MEC services in H-CRAN fiber wireless networks," *J. Lightw. Technol.*, vol. 38, no. 18, pp. 4935–4947, 2020.
[25] R. Gholivand and Z. Movahedi, "A cloud-RAN based end-to-end computation offloading in mobile edge computing," *Comput. Commun.*, vol. 175, pp. 193–204, 2021.
[26] S. Jeong, O. Simeone, and J. Kang, "Mobile edge computing via a UAV-mounted cloudlet: Optimization of bit allocation and path planning," *IEEE Trans. Veh. Technol.*, vol. 67, no. 3, pp. 2049–2063, Mar. 2018.
[27] Y. Shi, R. Enami, J. Wensowitch, and J. Camp, "Measurement-based characterization of LOS and NLOS drone-to-ground channels," in *Proc. IEEE Wireless Commun. Netw. Conf.*, 2018, pp. 1–6.
[28] C. Liu, M. Ding, C. Ma, Q. Li, Z. Lin, and Y.-C. Liang, "Performance analysis for practical unmanned aerial vehicle networks with LOS/NLOS transmissions," in *Proc. IEEE Int. Conf. Commun. Workshops*, 2018, pp. 1–6.
[29] Y. S. Meng and Y. H. Lee, "Measurements and characterizations of air-to-ground channel over sea surface at c-band with low airborne altitudes," *IEEE Trans. Veh. Technol.*, vol. 60, no. 4, pp. 1943–1948, May 2011.
[30] L. Bonati, S. D'Oro, M. Polese, S. Basagni, and T. Melodia, "Intelligence and learning in O-RAN for data-driven NextG cellular networks," *IEEE Commun. Mag.*, vol. 59, no. 10, pp. 21–27, Oct. 2021.
[31] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*. Cambridge, MA, USA: A Bradford Book, 2018.
[32] R. Porter, E. Nudelman, and Y. Shoham, "Simple search methods for finding a NASH equilibrium," *Games Econ. Behav.*, vol. 63, no. 2, pp. 642–662, 2008.
[33] V. François-Lavet, P. Henderson, R. Islam, M. G. Bellemare, and J. Pineau, "An introduction to deep reinforcement learning," *Found. Trends® Mach. Learn.*, vol. 11, no. 3/4, pp. 219–354, 2018.
[34] N. R. Devanur and T. P. Hayes, "The adwords problem: Online keyword matching with budgeted bidders under random permutations," in *Proc. 10th ACM Conf. Electron. Commerce*, 2009, pp. 71–78.
[35] Q. Shi, H. Sun, S. Lu, M. Hong, and M. Razaviyayn, "Inexact block coordinate descent methods for symmetric nonnegative matrix factorization," *IEEE Trans. Signal Process.*, vol. 65, no. 22, pp. 5995–6008, Nov. 2017.
[36] W. Zhong, R. Jin, C. Yang, X. Yan, Q. Zhang, and Q. Li, *Stock Constrained Recommendation in Tmall*. New York, NY, USA: Association for Computing Machinery, 2015, pp. 2287–2296. [Online]. Available: https://doi.org/10.1145/2783258.2788565
[37] S. Boyd and L. Vandenberghe, *Convex Optimization*, Cambridge, U.K.: Cambridge Univ. Press, 2004.
[38] T. Lattimore and C. Szepesvari, *Bandit Algorithms*, Cambridge, U.K.: Cambridge Univ. Press, 2020.
[39] R. Munos and A. Moore, "Variable resolution discretization in optimal control," *Mach. Learn.*, vol. 49, pp. 291–323, 2002.

**Chuan Pham** received the BS degree from the Hochiminh City University of Transport, the master's degree from the HoChiMinh City University of Sciences, and the PhD degree from Kyung Hee University, in electrical and computer engineering, in 2004, 2008 and 2017, respectively. He was a postdoc fellow with the Department of Computer Science and Engineering, Kyung Hee University, August, 2017. Since 2018, he has been a postdoc fellow in Synchromedia - École de Technologie Supérieure, Université du Québec.

**Foroutan Fami** received the masters's degree in information technology from Sharif University of Technology, Tehran, Iran, in 2016. He is currently a master student in telecommunication at École de technologie supérieure, Montreal, Canada. His research interests include wireless communications and network optimization.

**Kim Khoa Nguyen** received the PhD degree from Concordia University, in electrical and computer engineering. He is associate professor with the Department of Electrical Engineering, University of Quebec's Ecole de technologie supérieure, Montreal, Canada. In the past, he served as CTO of Inocybe Technologies (now is Kontron Canada), a leading company in software-defined networking (SDN) solutions. He was the architect of the Canarie's GreenStar Network and also involved in establishing CSA/IEEE standards for green ICT. He has led R&D in large-scale projects with Ericsson, Ciena, Telus, and InterDigital.

**Mohamed Cheriet** received the MSc and PhD degrees in computer science from the University of Pierre & Marie Curie (Paris VI), in 1985 and 1988 respectively. Since 1992, he has been a professor with the Systems Engineering Department, University of Quebec - École de Technologie Supérieure (ÉTS), Montreal, and was appointed full professor there, in 1998. He is the founder and director of Synchromedia Laboratory for multimedia communication in telepresence applications, since 1998.

▷ **For more information on this or any other computing topic, please visit our Digital Library at** www.computer.org/csdl.

Authorized licensed use limited to: National Yang Ming Chiao Tung University. Downloaded on October 19,2025 at 06:42:59 UTC from IEEE Xplore. Restrictions apply.